

이론 4장

# SQL의 이론적 기초

선언형 언어, 관계 연산, SQL 4분류, NULL

한국기술교육대학교 경영학부

경영데이터베이스및실습 (CCT770) · 2026학년도 2학기

# 이 장을 마치면

|      |                                      |
|------|--------------------------------------|
| 목표 ① | SQL이 선언형 언어인 이유를 절차형 언어와 비교해 설명한다    |
| 목표 ② | 선택·투영·조인·집합 연산이 어느 SQL 문법인지 짚는다      |
| 목표 ③ | DDL·DML·DCL·TCL의 역할을 구분하고 대표 명령을 말한다 |
| 목표 ④ | NULL이 끼인 비교·논리 연산을 3값 논리로 설명한다       |
| 목표 ⑤ | NULL로 인한 집계 왜곡을 찾아내고 실무 수칙으로 예방한다    |

경영 DB 이론 편람 4장 · 다운유통 사례로 배웁니다

# 이 장에서 다루는 네 가지

---

**1** SQL은 왜 「선언형」 언어인가

**2** 관계 연산 — SQL 문법의 뿌리

**3** SQL 언어의 4분류 — DDL·DML·DCL·TCL

**4** NULL과 3값 논리 — 모르는 값의 계산법

1. SQL은 왜 선언형 언어인가

# 어떻게(How)가 아니라 무엇을(What)

| 구분       | 절차형 언어 (예: Python)                                | 선언형 언어 (SQL)                                            |
|----------|---------------------------------------------------|---------------------------------------------------------|
| 지시하는 것   | 어떻게(How) — 반복문을 돌며 한 건씩 검사하는 절차                   | 무엇을(What) — 원하는 결과의 조건                                  |
| 실행 방법 결정 | 프로그래머가 코드로 직접 결정                                  | DBMS의 옵티마이저가 자동 결정                                      |
| 다운유통 예   | 「고객 목록을 처음부터 끝까지 읽어라. grade가 VIP면 name을 결과에 담아라.」 | SELECT name FROM customers WHERE grade = 'VIP' — 조건만 선언 |

택시에서 「시청까지 가 주세요」라고 목적지만 말하듯 — 어떤 순서로 뒤질지는 DBMS의 옵티마이저가 정한다

# 그래서 SQL은 전산 전공자만의 도구가 아니다

## 업무 질문을 조건으로

01

절차를 몰라도 「이번 달 매출은?」「품질 상품은?」을  
조건으로 옮길 줄만 알면 데이터에 직접 물을 수 있다

## 질문을 가장 잘 아는 사람이

02

직접 묻는 것이 보고서를 기다리는 것보다  
빠르고 정확하다 — 경영학부생이 SQL을 배우는 이유

SQL 문법별 실습은 3·5·7·9주차 강의에서 — 이 장은 그 문법들이 어떤 이론 위에 서 있는지 정리한다

## 2. 관계 연산

# 테이블에 연산을 가하면 결과도 테이블

### 선택

01

Selection

조건에 맞는 행 고르기

→ WHERE

### 투영

02

Projection

필요한 열 고르기

→ SELECT 열 목록

### 조인

03

Join

두 테이블 이어 붙이기

→ JOIN ... ON

### 집합 연산

04

결과끼리

합치고 빼기

→ UNION 등

SQL 문법은 관계 대수(Relational Algebra)에서 나왔다 — 결과가 또 표이므로 연산을 몇 번이고 이어 쓸 수 있다

## 선택 — 행을 고른다 (WHERE)

```
1 SELECT *
2 FROM   customers
3 WHERE  grade = 'VIP';
4 -- 고객 10명(행 10개) 중 조건에 맞는 한가람 · 임수빈 2행만 남는다.
5 -- 열은 7개 그대로 → 가로로 자른 것
```

선택(Selection) = 테이블을 가로로 잘라 조건에 맞는 행만 남긴다

## 투영 — 열을 고른다 (SELECT 열 목록)

```
1 SELECT name, grade
2 FROM   customers;
3 -- 열 7개 중 name · grade 2개만 남는다.
4 -- 행은 10개 그대로 → 세로로 자른 것
5
6 -- 실무 쿼리는 대부분 선택 + 투영의 조합이다
7 SELECT name, points      -- 투영: 열 2개만
8 FROM   customers
9 WHERE  grade = 'VIP';    -- 선택: VIP 행만
```

투영(Projection) = 테이블을 세로로 잘라 필요한 열만 남긴다

## 2. 관계 연산

# 조인 — 테이블을 잇는다 (JOIN ... ON)

```
1 SELECT o.order_id, c.name, o.order_date
2 FROM   orders o
3 JOIN   customers c ON o.cust_id = c.cust_id;
4 -- 주문 1001(C001) 행에 고객 한가람 행이 이어 붙어
5 -- "1001 | 한가람 | 2026-01-10" 처럼 읽히는 결과가 된다.
```

주문 테이블에는 고객번호만 있다 — 「누가 주문했는가」를 읽으려면 공통 값(기본키-외래키)으로 잇는다

2. 관계 연산

# 관계 연산과 SQL 문법의 대응

| 관계 연산 | 하는 일                   | 대응하는 SQL 문법           | 다운유통 예                              |
|-------|------------------------|-----------------------|-------------------------------------|
| 선택    | 조건에 맞는 행을 고른다 (가로 자르기) | WHERE                 | WHERE grade = 'VIP'                 |
| 투영    | 필요한 열만 남긴다 (세로 자르기)    | SELECT 열 목록           | SELECT name, grade                  |
| 조인    | 공통 값으로 두 테이블을 잇는다      | JOIN ... ON           | orders와 customers를 cust_id로 연결      |
| 합집합   | 두 결과를 합친다 (중복 제거)      | UNION                 | 오프라인 주문 고객 ∪ 온라인 주문 고객              |
| 교집합   | 두 결과의 공통만 남긴다          | INTERSECT (미지원 시 IN)  | 가전도 사고 식품도 산 고객                     |
| 차집합   | 한 결과에서 다른 결과를 뺀다       | EXCEPT (미지원 시 NOT IN) | 전체 고객 - 주문 고객 = 미주문 고객              |
| 카티션 곱 | 두 테이블의 모든 행 조합을 만든다    | CROSS JOIN            | customers 10행 × products 10행 = 100행 |

집합 연산은 열의 개수와 자료형이 서로 맞는 결과끼리만 — 전체 고객에서 주문 고객을 빼면 C010 서지안이 남는다

# 카티션 곱은 대부분 실수의 결과다

## ON을 빠뜨리면

01

고객 10명 × 주문 15건  
= 150행이 쏟아진다

## 실제 규모라면

02

회원 10만 명 × 주문 100만 건  
= 1,000억 행, 시스템이 멈춘다

## 점검 습관

03

결과 행 수가 예상보다  
훨씬 많다면  
조인 조건부터 의심

조인 조건(ON)을 빠뜨리면 모든 조합이 만들어져 결과가 폭발한다

3. SQL 언어의 4분류

DDL · DML · DCL · TCL

| 분류  | 이름       | 대표 명령                          | 하는 일            | 다운유통에서 누가 쓰나              |
|-----|----------|--------------------------------|-----------------|---------------------------|
| DDL | 데이터 정의어  | CREATE, ALTER, DROP            | 구조를 만들고·바꾸고·없앤다 | DB 관리자 — customers 테이블 생성 |
| DML | 데이터 조작어  | SELECT, INSERT, UPDATE, DELETE | 조회하고·넣고·고치고·지운다 | 실무 부서 — 주문 입력, 매출 조회      |
| DCL | 데이터 제어어  | GRANT, REVOKE                  | 권한을 주고 회수한다     | DB 관리자 — 영업팀에 조회 권한 부여    |
| TCL | 트랜잭션 제어어 | COMMIT, ROLLBACK               | 변경을 확정하거나 되돌린다  | 주문 처리 프로그램 — 5장에서 자세히     |

구조를 바꾸는 명령과 데이터를 조회하는 명령은 위험도가 전혀 다르다 — 그래서 사람마다 쓸 수 있는 갈래도 다르다

# 현업은 DML, 통제는 DCL

## 현업이 쓰는 것 — SELECT

01

구조(DDL)와 권한(DCL)은 관리자의 영역  
현업의 일은 「데이터에 묻는 것」 — 실습 대부분이 SELECT

## 내부통제 — DCL

02

「인사팀만 급여 열을 본다」「아르바이트는 조회만」  
회계감사의 접근 권한 통제가 기술적으로는 DCL

DCL의 자세한 내용은 8장에서 다룬다

4. NULL과 3값 논리

# NULL은 0도, 빈 문자열도 아니다

| 위치                          | 값    | NULL의 의미                             |
|-----------------------------|------|--------------------------------------|
| employees.salary (E008 조하늘) | NULL | 아직 모름 — 연봉 협상 중이라 값이 정해지지 않았다        |
| orders.emp_id (온라인 주문)      | NULL | 해당 없음 — 온라인 주문에는 담당 직원이 원래 없다        |
| customers.email (C005 박서연)  | NULL | 미입력 — 고객이 이메일을 등록하지 않았다              |
| customers.points (C010 서지안) | 0    | NULL이 아니다! 0점이라고 알고 있는 것과 모르는 것은 다르다 |

NULL = 「값이 없음」을 나타내는 특별한 표시 — 「아직 모름」 또는 「해당 없음」이라는 상태 그 자체

#### 4. NULL과 3값 논리

## NULL 비교는 = 가 아니라 IS NULL

```
1  -- 잘못된 방법: 결과가 0건 (비교 결과가 전부 '모름')
2  SELECT * FROM orders WHERE emp_id = NULL;
3
4  -- 올바른 방법: 온라인 주문 7건이 조회된다
5  SELECT * FROM orders WHERE emp_id IS NULL;
6
7  -- 반대는 IS NOT NULL: 담당 직원이 있는 오프라인 주문
8  SELECT * FROM orders WHERE emp_id IS NOT NULL;
```

모르는 값과는 비교가 성립하지 않는다 — NULL = NULL 조차 참이 아니라 「모름(UNKNOWN)」

# 3값 논리 조합표 — 참·거짓·모름

| 조건 A   | 조건 B | A AND B      | A OR B         |
|--------|------|--------------|----------------|
| 참      | 모름   | 모름           | 참 (하나가 참이면 충분) |
| 거짓     | 모름   | 거짓 (이미 결정)   | 모름             |
| 모름     | 모름   | 모름           | 모름             |
| NOT 모름 | —    | 모름 (부정해도 모름) | —              |

결과를 뒤집을 가능성이 남아 있으면 모름, 이미 결정됐으면 그 값

WHERE 절은 결과가 '참'인 행만 통과시킨다 — 거짓인 행뿐 아니라 모름인 행도 걸러진다

#### 4. NULL과 3값 논리

## NULL이 경영 수치를 왜곡하는 장면

```
1  -- 직원은 8명이지만 salary가 있는 직원은 7명 (E008 조하늘 NULL)
2  SELECT COUNT(*)      AS 전체직원수,    -- 8 (행 수를 센다)
3          COUNT(salary) AS 연봉등록수,    -- 7 (NULL을 빼고 센다)
4          AVG(salary)   AS 평균연봉      -- 7명의 평균 (NULL은 분모에서도 제외)
5  FROM    employees;
6
7  -- 함정: "연봉 400 미만 직원" + "연봉 400 이상 직원" ≠ 전체 직원
8  SELECT COUNT(*) FROM employees WHERE salary < 400; -- 4명
9  SELECT COUNT(*) FROM employees WHERE salary >= 400; -- 3명
10 -- 4 + 3 = 7명. 조하늘은 어느 쪽 조건도 '참'이 아니라 양쪽 모두에서 빠졌다!
```

구간별 합계가 전체와 맞지 않으면 — 대부분 NULL인 행이 모든 구간에서 빠졌기 때문

#### 4. NULL과 3값 논리

# NULL 실무 수칙 다섯 가지



## NULL 검사는 IS NULL / IS NOT NULL

= NULL 은 항상 「모름」이 되어 0건이다



## COUNT(\*)와 COUNT(열)을 구분

둘의 차이가 곧 NULL의 개수다



## 구간 집계에는 「미정」 구간

NULL 행은 어디에도 속하지 않는다



## 보고서에는 IFNULL로 읽기 좋게

IFNULL(salary, 0), IFNULL(e.name, '온라인')



## 입력 단계에서 막는 것이 최선

꼭 있어야 하는 값이면 NOT NULL 제약(3장)

숫자가 맞지 않으면 가장 먼저 NULL을 의심한다

# 확인 퀴즈

다온유통에서 다음 네 작업을 맡은 명령을 SQL 4분류로 짝지을 때 옳은 것은? (가) customers 테이블 생성 (나) VIP 고객 명단 조회 (다) 영업팀에 조회 권한 부여 (라) 주문 처리 변경을 확정

① (가) DDL (나) DML (다) DCL (라) TCL

② (가) DDL (나) DML (다) TCL (라) DCL

③ (가) DML (나) DDL (다) DCL (라) TCL

④ (가) DDL (나) DCL (다) DML (라) TCL

## 정답 및 해설

### 정답

① (가) DDL (나) DML (다) DCL (라) TCL

### 해설

CREATE로 구조를 만드는 것은 데이터 정의어(DDL), SELECT로 데이터를 조회하는 것은 데이터 조작어(DML), GRANT로 권한을 주는 것은 데이터 제어어(DCL), COMMIT으로 변경을 확정하는 것은 트랜잭션 제어어(TCL)입니다. 경영학부 출신이 실무에서 가장 많이 쓰는 것은 DML의 SELECT이고, DDL과 DCL은 대체로 DB 관리자의 영역입니다.

# 핵심 요약

|      |                                                  |
|------|--------------------------------------------------|
| 핵심 ① | SQL은 결과(What)를 선언 — 실행 방법은 옵티마이저가 정한다            |
| 핵심 ② | 선택=WHERE, 투영=SELECT 열 목록, 조인=JOIN ON, 집합=UNION 등 |
| 핵심 ③ | 조인 조건을 빠뜨리면 카티션 곱 — 결과가 폭발한다                     |
| 핵심 ④ | DDL·DML·DCL·TCL — 현업이 주로 쓰는 것은 DML의 SELECT       |
| 핵심 ⑤ | NULL은 「값 없음」, NULL이 낀 비교는 「모름」(3값 논리)            |
| 핵심 ⑥ | WHERE는 참인 행만 통과 — 구간 합계가 안 맞으면 NULL부터            |

문법 하나하나가 관계 연산과 3값 논리라는 이론 위에 서 있다

다음 장 예고

# 이론 5장 · 트랜잭션과 동시성

01

## 트랜잭션

쪼갤 수 없는  
업무 한 묶음

02

## ACID

트랜잭션이 지켜야 할  
4가지 성질

03

## 확정과 취소

COMMIT과  
ROLLBACK

04

## 동시성과 회복

락과 격리 수준  
장애 회복

이론 4장 마무리

## 학습 마무리

주차별 SQL 실습에서 오늘의 이론을 문법으로 다시 만납니다

✓ 선언형 언어와 옵티마이저

✓ SQL 4분류 — DDL·DML·DCL·TCL

✓ 관계 연산과 SQL 문법의 대응

✓ NULL과 3값 논리, 집계 함정

관련 주차

3·5·7·9주차 강의에서 SQL로 확인